

Barbearia Primordial

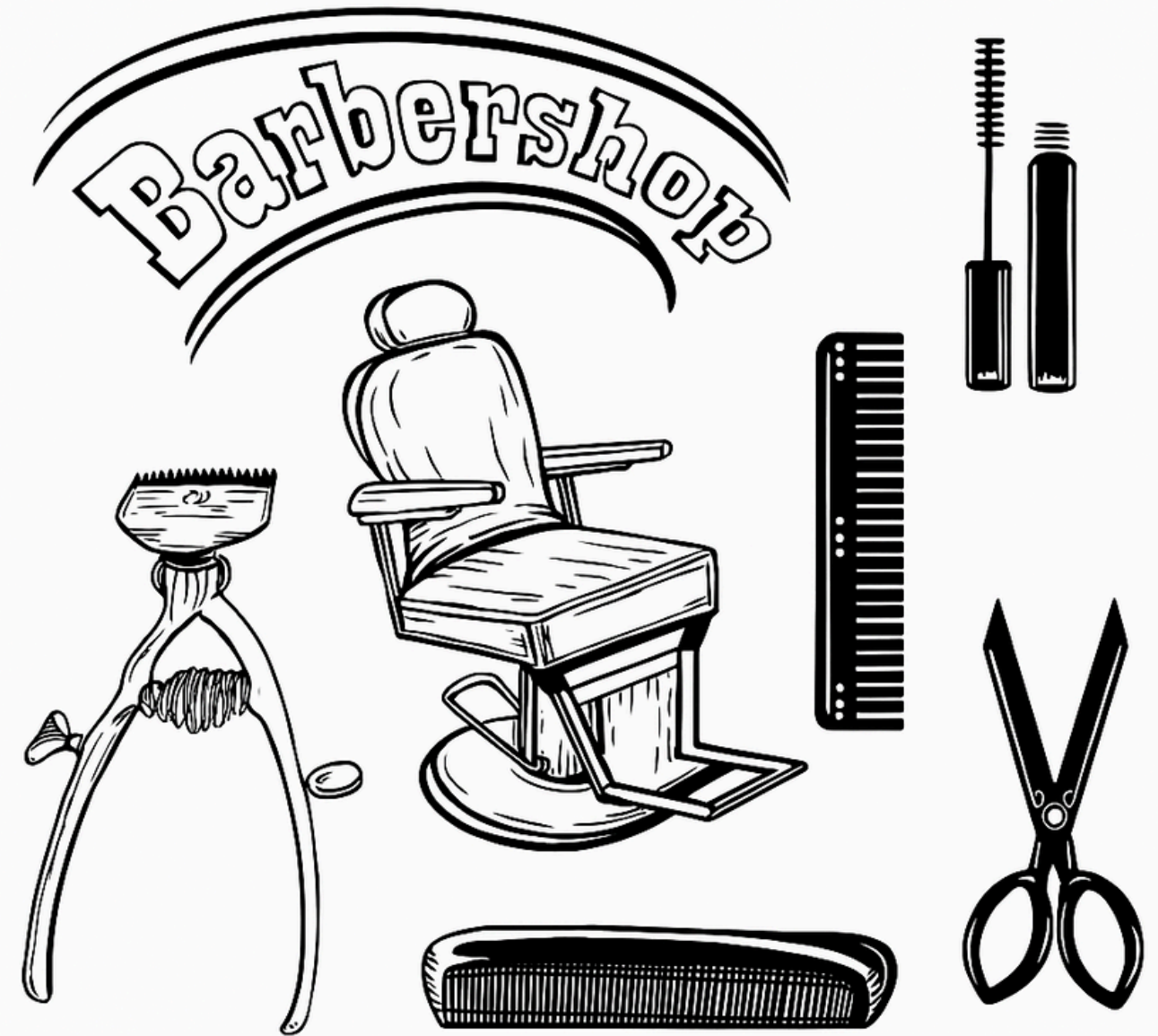
Projeto de extensão do 2º semestre

Integrantes:

- Isabela Yumi Nitta - CP3035719
- Larissa Prado Santana - CP3035387
- Matheus Martins Souza - CP3037029
- Rafaela Ágatha Souza Camargo - CP3035549

Sumário

- 01 Objetivo
- 02 Justificativa
- 03 Problema
- 04 Conceitos e ferramentas
- 05 Modelagem de Software (ESW)
- 06 Estrutura do código (LP2)
- 07 Implementação prática do projeto
- 08 Banco de dados
- 09 Principais dificuldades encontradas
- 10 Conclusão



1. OBJETIVO



Desenvolvimento de um sistema digital de programação para uma barbearia;



Acessível aos profissionais e gestores do estabelecimento.

2. JUSTIFICATIVA



Melhorar o atendimento;



Oferecer praticidade e rapidez;



Facilitar a rotina da barbearia.

3. PROBLEMA



A barbearia realiza agendamentos manualmente.

- Consome muito tempo;
- Aumenta chance de erros.

4. CONCEITOS E FERRAMENTAS

- ✓ Linguagem C# (.NET framework);
- ✓ Banco de dados (MySQL);
- ✓ Diagramas de caso e classe;
- ✓ Prototipação.

5. MODELAGEM DE SOFTWARE (ESW)

- Captura de Requisitos do Software (técnica utilizada e o porquê);
- Identificação dos usuários;
- Identificação de requisitos funcionais e não funcionais;
- Diagrama de Caso de Uso;
- Diagrama de Classes;
- Protótipo x Versão final.

Captura de Requisitos do Software



Técnica: entrevista



Conversas diretas com o dono da barbearia para levantar as necessidades do negócio



Entendimento a fundo sobre o fluxo de trabalho da barbearia



Identificação dos usuários

Ator (Usuário)	Descrição e responsabilidades no sistema
Administrador	Acesso total ao sistema, responsável pela gestão de usuários (criar/definir permissões), visualização de relatórios, análises e realização de backup de dados. Também gerencia a parte financeira.
Funcionário	Responsável pelas operações diárias, como registrar vendas, realizar o controle de clientes e estoque, gerenciar serviços.
Cliente	Interage com o sistema para monitoramento, com a fila de espera, podendo visualizar sua posição e o tempo de espera

Requisitos Funcionais

	Nome do requisito	RNF associados
RF-01	Cadastro de identificação do cliente	RNF-01, RNF-03, RNF-06
RF-02	Cadastro da posição na lista de espera	RNF-03, RNF-06
RF-03	Cadastro do serviço a ser prestado	RNF-03, RNF-06
RF-04	Cadastro do funcionário que realizará o serviço	RNF-03, RNF-06
RF-05	Cadastro de indicação de compra de produtos a parte	RNF-03, RNF-06
RF-06	Cadastro de produtos escolhidos pelo cliente	RNF-03, RNF-06
RF-07	Finalização do cadastro	RNF-02, RNF-03, RNF-06
RF-08	Pesquisar cliente	RNF-01, RNF-02, RNF-03, RNF-06
RF-09	Remover cliente	RNF-03, RNF-06, RNF-10
RF-10	Editar cadastro do cliente	RNF-03, RNF-06, RNF-10
RF-11	Relatório de cadastros	RNF-03, RNF-06
RF-12	Fechar do programa	RNF-03, RNF-06

Requisitos Não funcionais

	Categoria	Descrição do Requisito
RNF-01	Segurança	Garantir a confidencialidade das informações pessoais.
RNF-02	Desempenho	O tempo de resposta deve ser inferior a 5 segundos.
RNF-03	Usabilidade	A interface deve ser amigável e intuitiva.
RNF-04	Confiabilidade	Capaz de recuperar dados do banco em caso de falhas de energia ou interrupções, garantindo zero perdas.
RNF-05	Padrões	Padrões de nomenclatura do C# e as melhores práticas de Orientação a Objetos.
RNF-06	Hardware e Software	Compatível com Windows 10 ou superior e requer hardware mínimo de 4GB de RAM e 10GB de disco.
RNF-07	Organizacional	Concluído e entregue até o início de dezembro.
RNF-08	Legais	Estar em conformidade com as leis de proteção de dados vigentes (LGPD).
RNF-09	Entrega	Incluir um manual de instalação e um guia rápido de uso para o funcionário.
RNF-10	Privacidade	Possuir um controle de acesso para funcionários, exigindo uma autenticação (como login e senha) antes de permitir qualquer ação

Diagrama de caso de uso

- Identificação dos atores, funcionalidades e relacionamentos especiais (Include e Extend)

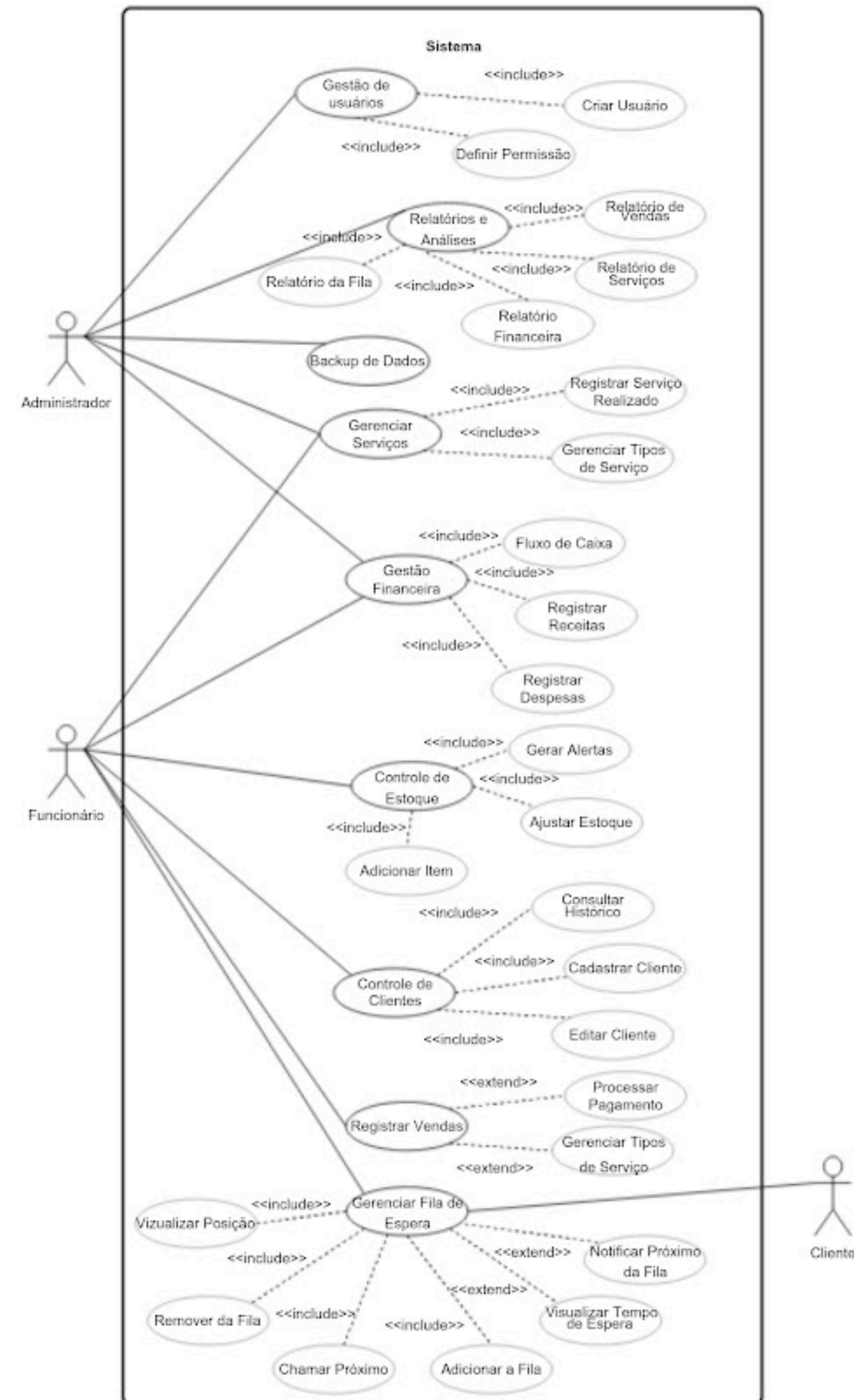
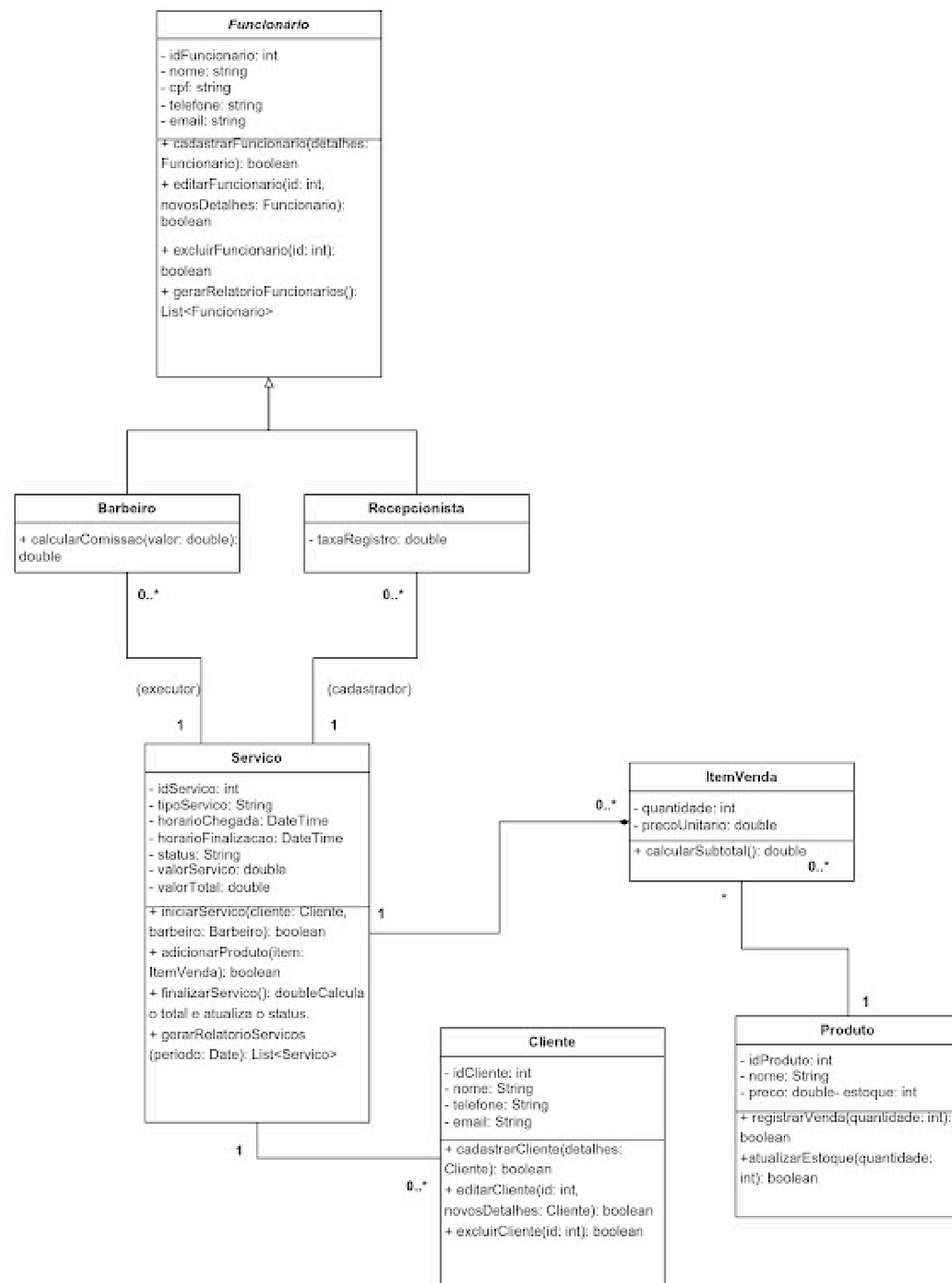


Diagrama de classes

- Representação da estrutura, seus dados (atributos) e comportamentos (métodos).



Protótipo

- Versão inicial das telas mais significativas do sistema (cadastro do cliente)

Cadastro Barbearia Primordial

Cadastro

Insira o CPF do cliente: 345.678.234-87

Insira o NOME do cliente: João

Insira o TELEFONE do cliente: (19) 920215613

Próximo...

Cadastro Barbearia Primordial

Cadastro

Selecione o funcionário a prestar o serviço marcado anteriormente:

- ☐ Tavin
- ☐ Theo
- ☐ Maillon
- ☒ Maycon
- ☐ Vinicios

Voltar Próximo...

Cadastro Barbearia Primordial

Cadastro

Insira o horário de chegada do cliente
(formato: 12h00 ou 12:00):

13h40

Voltar Próximo...

Cadastro Barbearia Primordial

Cadastro

Selecione o serviço a ser prestado:

- ☐ Corte:.....(R\$35,00)
- ☒ Corte + sobancelha:.....(R\$45,00)
- ☐ Corte e barba:.....(R\$60,00)
- ☐ Corte, barba e sobancelha:...(R\$70,00)

Voltar Próximo...

Versão final

- Mudança de cores;
- Junção de todas as telas em uma só;
- Adição da data desejada.

Cadastro (info. cliente) Barbearia Primordial

Barbearia Primordial

Cadastro do cliente

CPF:

Nome:

Telefone:

Data marcada: 27/11/2025 

Horário marcado: 12:34:55 

Serviço desejado

- ☐ Corte:.....R\$35,00
- ☐ Corte + Sobancelha:.....R\$35,00
- ☐ Corte + Barba:.....R\$35,00
- ☐ Corte + Barba + Sobancelha:.....R\$35,00

Funcionário disponível

- ☐ Tavin
- ☐ Theo
- ☐ Maillon
- ☐ Maycon
- ☐ Vinicios

Voltar **Cancelar** **Próximo**

Protótipo

Esta imagem mostra um protótipo de uma interface de usuário para um sistema de cadastro. A janela principal, intitulada "Cadastro Barbearia Primordial", possui um fundo vermelho escuro. No topo, há uma barra de título com o ícone do Windows e o texto "Cadastro Barbearia Primordial", além de botões de minimizar, maximizar e fechar. O conteúdo principal é um formulário branco centralizado com o título "Cadastro" em negrito. Abaixo dele, a pergunta "O cliente vai comprar algum produto da loja?" é seguida por duas opções de radio button: "Sim" (selecionada) e "Não". Na base do formulário, há dois botões: "Voltar" e "Próximo...".

Cadastro Barbearia Primordial

Cadastro

O cliente vai comprar algum produto da loja?

☒ Sim ☐ Não

Voltar Próximo...

Versão final

Esta imagem apresenta a versão final da interface de usuário. A janela, agora intitulada "Cadastro (Escolha compra) Barbearia Primordial", mantém o tema de cores, mas com uma paleta mais sofisticada. O formulário centralizado possui um fundo marrom escuro e o título "Cadastro" em uma caixa de texto destacada. A pergunta "O cliente irá comprar algo da loja?" é apresentada em uma cor clara. As opções de radio button "Sim" e "Não" também são claras. Na base, os botões "Voltar", "Cancelar" e "Próximo" são mais proeminentes, com bordas brancas e fundo marrom.

Cadastro (Escolha compra) Barbearia Primordial

Cadastro

O cliente irá comprar algo da loja?

☒ Sim ☐ Não

Voltar Cancelar Próximo

Protótipo

Cadastro Barbearia Primordial

Barbearia Primordial

Cadastro

Categoria:
Roupa

Produto:
Blusa

Quantidade:
2

Tamanho:
☐ P ☒ M ☐ G ☐ GG

Adicionar

Voltar **Próximo...**

- A interface atual é dinâmica: as opções de tamanho (ml ou P/M/G) e a lista de produtos se adaptam automaticamente à categoria selecionada.

Versão final

Cadastro (Produtos) Barbearia Primordial

Barbearia Primordial

Cadastro

Categoria:
Produto de cabelo

Produto:
☐ Gel
☐ Creme
☐ Oléo

Tamanho:
☐ 50ml ☐ 100ml ☐ 200ml

Quantidade:
0

Adicionar

Voltar **Cancelar** **Finalizar**

Cadastro (Produtos) Barbearia Primordial

Barbearia Primordial

Cadastro

Categoria:
Roupa

Produto:
☐ Calça cargo
☐ Blusa estampada caveira
☐ Jaqueta

Tamanho:
☐ P ☐ M ☐ G ☐ GG

Quantidade:
0

Adicionar

Voltar **Cancelar** **Finalizar**

6. ESTRUTURA DO CÓDIGO (LP2)

- Interface gráfica (WindowsForms);
- Classe (campos, modificadores de acesso: public e private, construtor, Getters e Setters, método com passagem e retorno de valor;);
- Uso de referência;
- Tratamento de erros por meio de exceções;
- Acesso ao banco de dados MySql.

7. IMPLEMENTAÇÃO PRÁTICA DO PROJETO



Interface gráfica (WindowsForms);
Menu principal com opções funcionais.



Classe cliente

- Campos criados e modificadores de acesso: public e private;
- Construtor

```
9      51 referências
10     public class Cliente
11     {
12         private string nome;
13         private string cpf;
14         private string telefone;
15         private string servicoEscolhido;
16         private string funcioEscolhido;
17         private DateTime horarioMarcado;
18         private DateTime dataMarcada;
19         private List<Compra> compras;
20
21     2 referências
22     public Cliente()
23     {
24         this.nome = "";
25         this.cpf = "000000000000";
26         this.telefone = "0000000000";
27         this.servicoEscolhido = "";
28         this.funcioEscolhido = "";
29         this.horarioMarcado = DateTime.Now;
30         this.dataMarcada = DateTime.Now;
31         this.compras = new List<Compra>();
32     }
```

Classe cliente

- Métodos Getters e Setters no nome e nos demais dados do cliente

30	↓	3 referências
31	↓	<code>public string getNome()</code>
32	↓	<code>{</code>
33	↓	<code> return this.nome;</code>
	↓	<code>}</code>
34	↓	2 referências
35	↓	<code>public void setNome(string nome)</code>
36	↓	<code>{</code>
37	↓	<code> this.nome = nome;</code>
38	↓	<code>}</code>

Classe cadastro

- Método com passagem e retorno de valores;

```
49  ↓
50  ↓
51  ↓
52  ↓
53  ↓
54  ↓
55  ↓
56  ↓
57  ↓
58  ↓
59  ↓
60  ↓
61  ↓
62  ↓
63  ↓
64  ↓
65  ↓
66  ↓
67  ↓
68  ↓
69  ↓
70  ↓
71  ↓
72  ↓
73  ↓

0 referências
public bool EditarCliente(string cpf, Cliente novosDados)
{
    Cliente cliente = PesquisarClientePorCPF(cpf);

    if (cliente != null)
    {
        if (novosDados.getCpf() != cpf)
        {
            if (ClienteDAO.CPFExiste(novosDados.getCpf()))
            {
                throw new ArgumentException("CPF já cadastrado em outro cliente.");
            }
        }

        cliente.setNome(novosDados.getNome());
        cliente.setTelefone(novosDados.getTelefone());
        cliente.setServicoEscolhido(novosDados.getServicoEscolhido());
        cliente.setFuncioEscolhido(novosDados.getFuncioEscolhido());
        cliente.setHorarioMarcado(novosDados.getHorarioMarcado());
        cliente.setDataMarcada(novosDados.getDataMarcada());

        return ClienteDAO.Atualizar(cliente);
    }
    return false;
}
```

Uso de referência

- Ao clicar no botão, cria-se uma nova instância da janela de cadastro.
- Passa o objeto cadastro como parâmetro, garantindo que a nova janela tenha acesso à referência dos dados.

```
--  
23 1 referência  
24 private void btnCadastro_Click(object sender, EventArgs e)  
25 {  
26     FrmTipoCadastro frmCadastro = new FrmTipoCadastro(cadastro);  
27     frmCadastro.ShowDialog();  
28 }
```


Tratamento de erros por meio de exceções

- Método Convert

2 referências

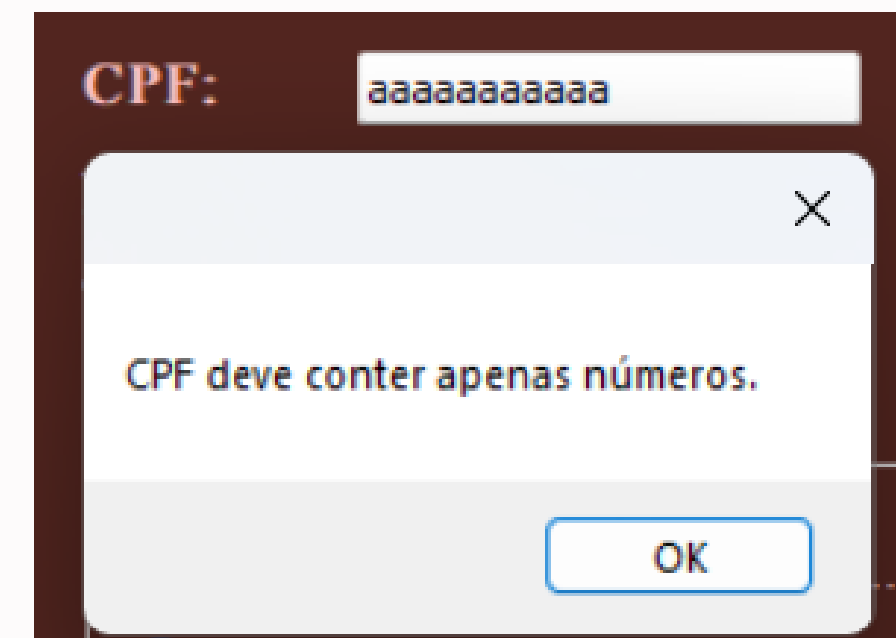
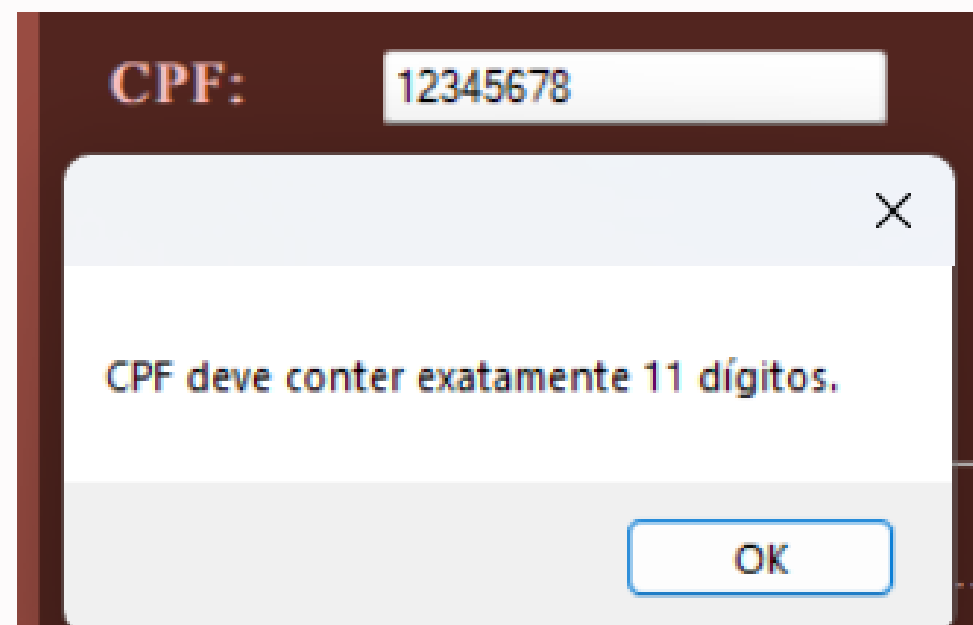
```
485  public static List<Estoque> ListarTodos()
486  {
487      try
488      {
489          string sql = "SELECT * FROM produto ORDER BY nome_produto";
490          DataTable dt = ConexaoDB.ExecutarConsulta(sql);
491
492          List<Estoque> produtos = new List<Estoque>();
493          foreach (DataRow row in dt.Rows)
494          {
495              Estoque produto = new Estoque();
496              produto.setNome(row["nome_produto"].ToString());
497              produto.setDescricao(row["descricao"]?.ToString() ?? "");
498              produto.setPreco(Convert.ToDouble(row["preco"]));
499              produto.setQuantd(Convert.ToInt32(row["estoque"]));
500              produtos.Add(produto);
501          }
502
503          return produtos;
504      }
505      catch (Exception ex)
506      {
507          throw new Exception($"Erro ao listar produtos: {ex.Message}");
508      }
509  }
```



Tratamento de erros por meio de exceções

- Comando throw

```
43      2 referências  
44      public void setCpf(string cpf)  
45      {  
46          if (cpf.Length != 11)  
47          {  
48              throw new ArgumentException("CPF deve conter exatamente 11 dígitos.");  
49          }  
50          if (!cpf.All(char.IsDigit))  
51          {  
52              throw new ArgumentException("CPF deve conter apenas números.");  
53          }  
54          this.cpf = cpf;  
55      }
```



Acesso ao banco de dados MySql

- Bibliotecas ADO.NET: System.Data e MySql.Data.
- Objeto MySqlConnection para abrir o banco.
- MySqlCommand e ExecuteNonQuery para inserção, atualização e remoção.

```
1  using MySql.Data.MySqlClient;  
2  using System;  
3  using System.Collections.Generic;  
4  using System.Data;
```

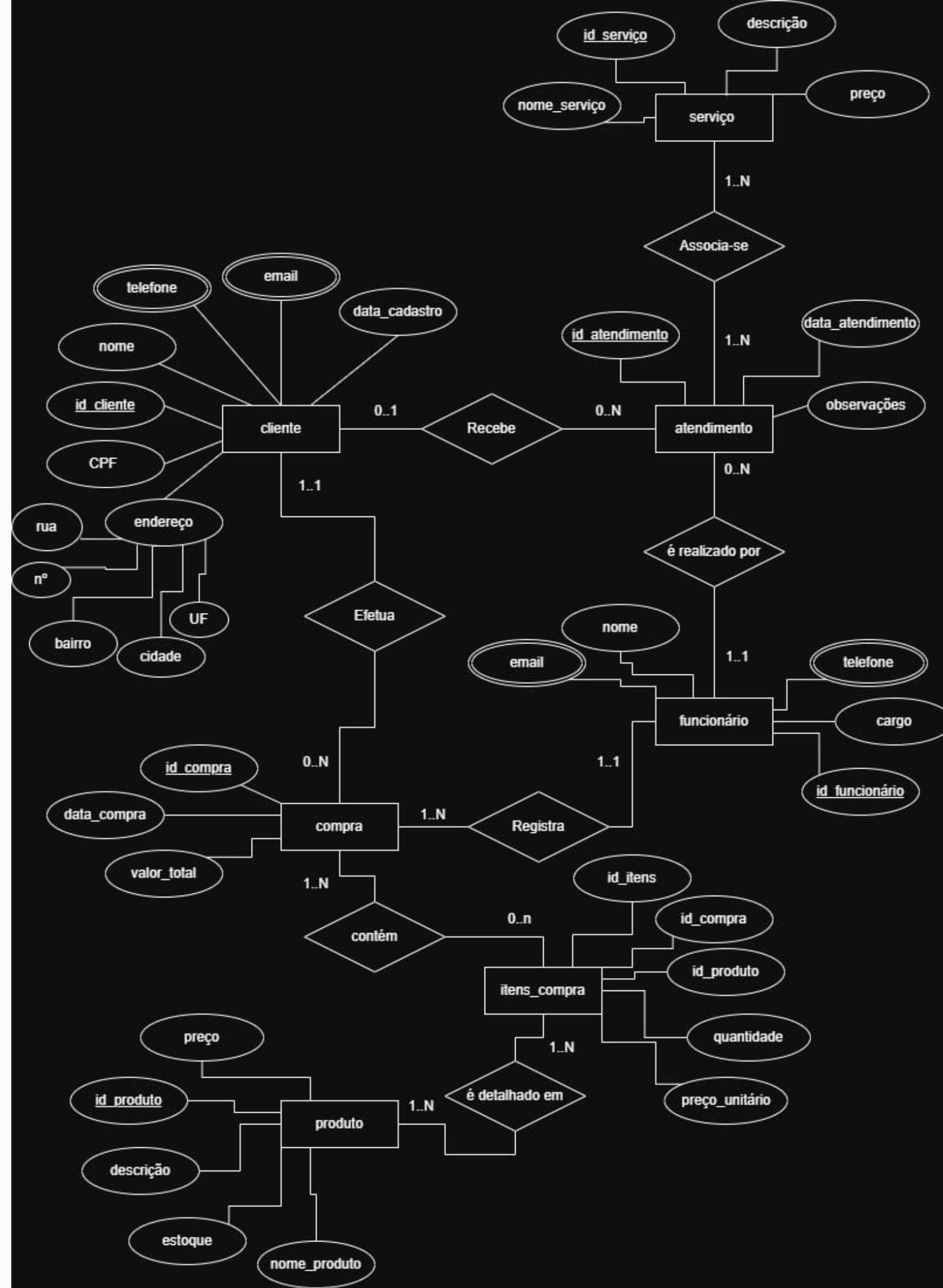
```
30 5 referências  
31 public static MySqlConnection AbrirConexao()  
32 {  
33     try  
34     {  
35         MySqlConnection conexao = new MySqlConnection(connectionString);  
36         conexao.Open();  
37         return conexao;  
38     }  
39     catch (MySqlException ex)  
40     {  
41         throw new Exception($"Erro ao conectar ao banco de dados: {ex.Message}");  
42     }  
43 }
```

```
78 4 referências  
79 public static int ExecutarComando(string sql, MySqlParameter[] parametros = null)  
80 {  
81     MySqlConnection conexao = null;  
82     try  
83     {  
84         conexao = AbrirConexao();  
85         using (MySqlCommand cmd = new MySqlCommand(sql, conexao))  
86         {  
87             if (parametros != null)  
88             {  
89                 cmd.Parameters.AddRange(parametros);  
90             }  
91             return cmd.ExecuteNonQuery();  
92         }  
93     }  
94     catch (MySqlException ex)  
95     {  
96         throw new Exception($"Erro ao executar comando: {ex.Message}");  
97     }  
98     finally  
99     {  
100        FecharConexao(conexao);  
101    }
```

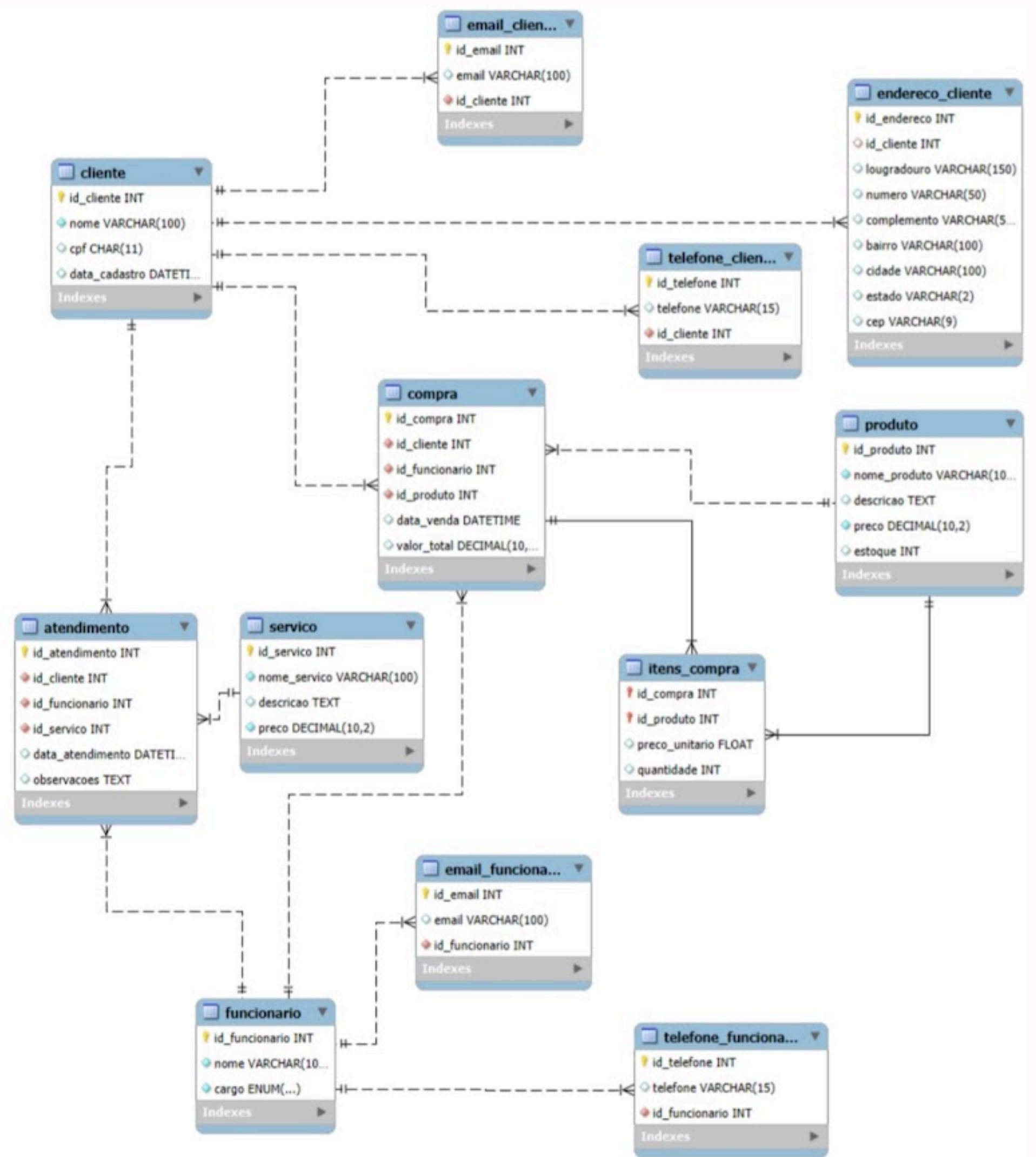
8. BANCO DE DADOS

- Modelo Entidade-Relacionamento (MER);
- Modelo Relacional (MRel);
- Análise das relações quanto às Formas Normais (FN);
- Álgebra Relacional e Consulta.

Modelo Entidade Relacionamento (MER)



Modelo Relacional (MRel)



Análise das relações quanto às Formas Normais (FN)

Forma Normal	Status	Justificativa no Projeto
1FN (Atomicidade)	✓ Atendido	Tabelas de Cliente e Funcionário decompostas para separar telefones, e-mails e endereços.
2FN (Dep. Parcial)	✓ Atendido	Tabela itens_compra (chave composta) tem seus atributos dependendo da chave inteira, não de partes.
3FN (Dep. Transitiva)	✓ Atendido	Uso correto de Chaves Estrangeiras: dados do cliente não se repetem na tabela de vendas.
FNBC (Determinantes)	✓ Atendido	Todos os determinantes identificados são chaves candidatas.

Álgebra Relacional e Consulta

Objetivo: Identificar o nome do funcionário que realizou o maior número de atendimentos, e o total de atendimentos dele.

Álgebra Relacional:

ρ f, contagem (id_atendimento) $\rightarrow$ total_atendimento (funcionario $\bowtie$ atendimento)
 π f.nome, total_atendimentos (f | $\forall f': f'.total_atendimentos \leq f.total_atendimentos$)
Símbolo: $\forall$ (“para todo” ou “para cada”)

Consulta SQL:

```
SELECT F.nome AS Nome_Funcionario, F.cargo AS Cargo,  
COUNT(A.id_atendimento) AS Total_Atendimentos  
FROM funcionario F  
JOIN atendimento A ON F.id_funcionario = A.id_funcionario  
GROUP BY F.id_funcionario, F.nome, F.cargo  
ORDER BY Total_Atendimentos DESC  
LIMIT 1;
```

9. PRINCIPAIS DIFICULDADES ENCONTRADAS



Passagem de parâmetros entre interfaces: manter a consistência dos dados ao navegar entre as diferentes telas;



Equilibrar as funcionalidades planejadas com o prazo de entrega disponível.

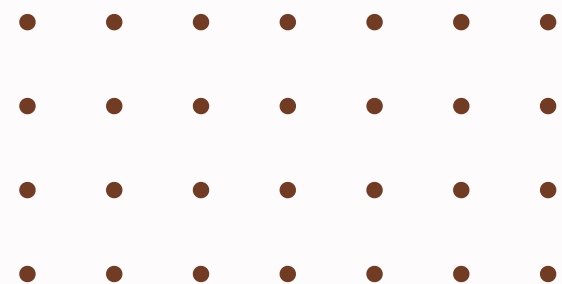
10. Conclusão



Atende à necessidade de modernização;



É eficaz e contribui positivamente na rotina da barbearia.



Obrigado!

Professores:

- Danilo Cesar Pereira
- Henrique Goncalves Salvador
- Fabio Feliciano de Oliveira
- Zady Castaneda Salazar